Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики і програмної інженерії

Звіт
з дисципліни «Технології розподілених обчислень»
Комп'ютерний практикум №4
«Розробка паралельних програм з використанням пулів потоків, екзекьюторів та ForkJoinFramework»

Виконав:
Студент III курсу
гр. ІП-33
Соколов О. В.

Прийняв:
Дифучин А. Ю.
"4" квітня 2026 р.

Київ – 2026

КОМП’ЮТЕРНИЙ ПРАКТИКУМ №4

Тема: Розробка паралельних програм з використанням пулів потоків, екзекьюторів та ForkJoinFramework

Завдання:

1. Побудуйте алгоритм статистичного аналізу тексту та визначте характеристики випадкової величини «довжина слова в символах» з використанням ForkJoinFramework. За основу візьміть код з джерела https://www.oracle.com/technical-resources/articles/java/fork-join.html . 20 балів. Дослідіть побудований алгоритм аналізу текстових документів на ефективність експериментально. 10 балів.
2. Реалізуйте один з алгоритмів комп’ютерного практикуму 2 або 3 з використанням ForkJoinFramework та визначте прискорення, яке отримане за рахунок використання ForkJoinFramework. 20 балів.
3. Розробіть та реалізуйте алгоритм пошуку спільних слів в текстових документах з використанням ForkJoinFramework. 20 балів.
4. Розробіть та реалізуйте алгоритм пошуку текстових документів, які відповідають заданим ключовим словам (належать до області «Інформаційні технології»), з використанням ForkJoinFramework. 30 балів.

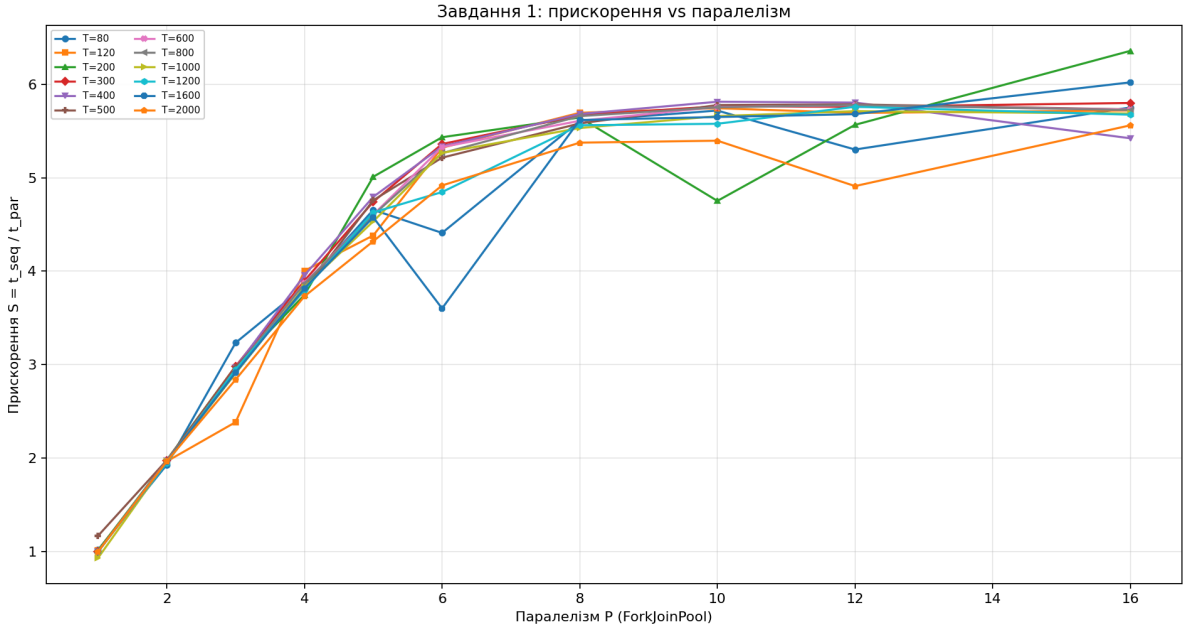
ВИКОНАННЯ

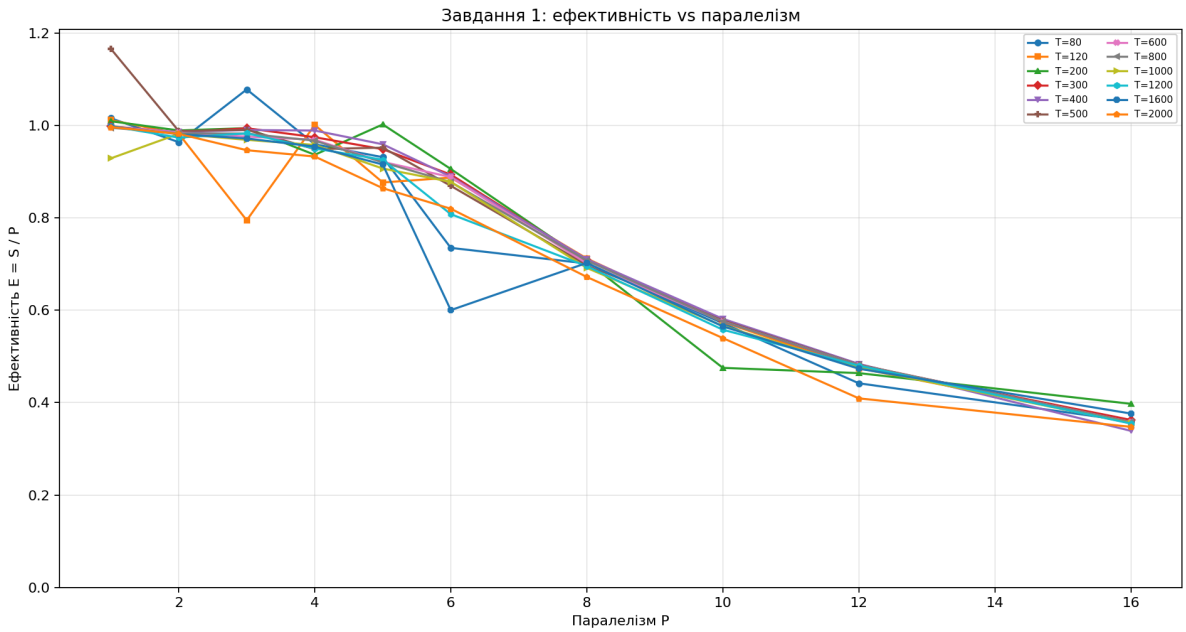
Завдання 1

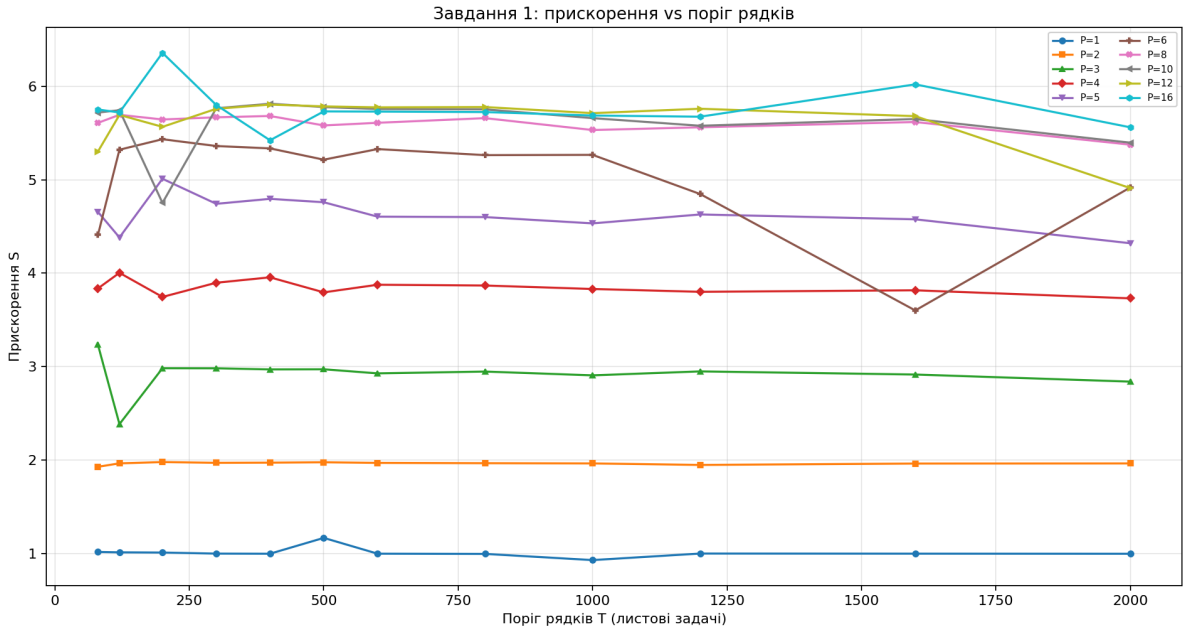
Потрібно оцінити випадкову величину «довжина слова в символах» за допомогою fork/join: середнє, середньоквадратичне відхилення, гістограма частот, плюс експериментально – прискорення і ефективність паралельного варіанту порівняно з послідовним.

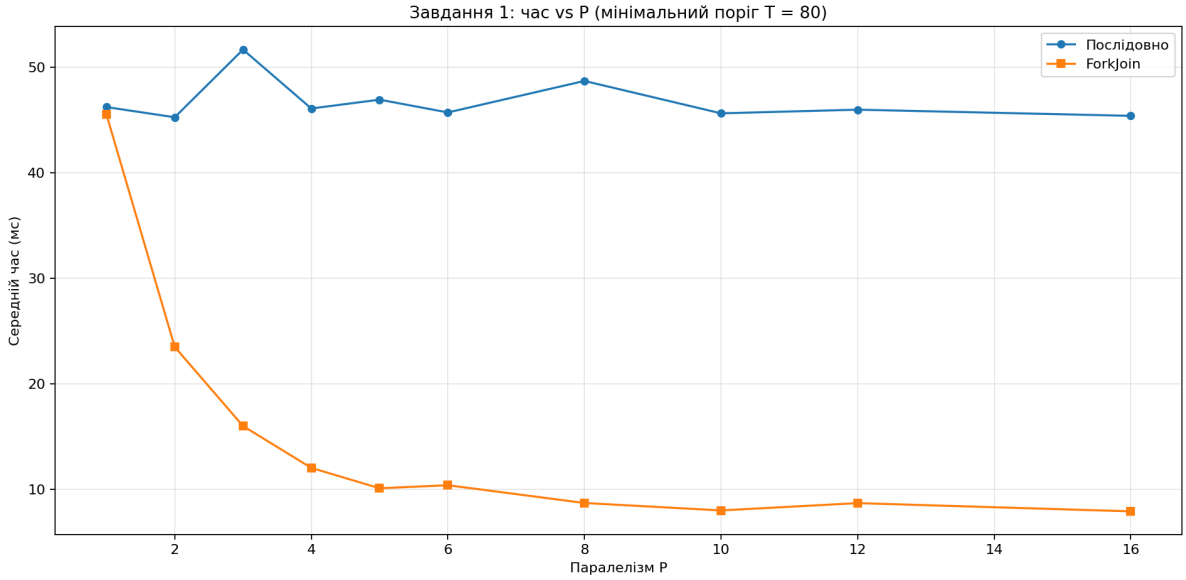
У експерименті порівнюють час послідовної та паралельної версій на тих самих даних, з розігрівом і повторними прогонами. Зручно варіювати кількість потоків і грубість розбиття (як великі фрагменти стають «листям» дерева задач): занадто великі – гірше заповнюються потоки, занадто дрібні – ростуть накладні витрати.

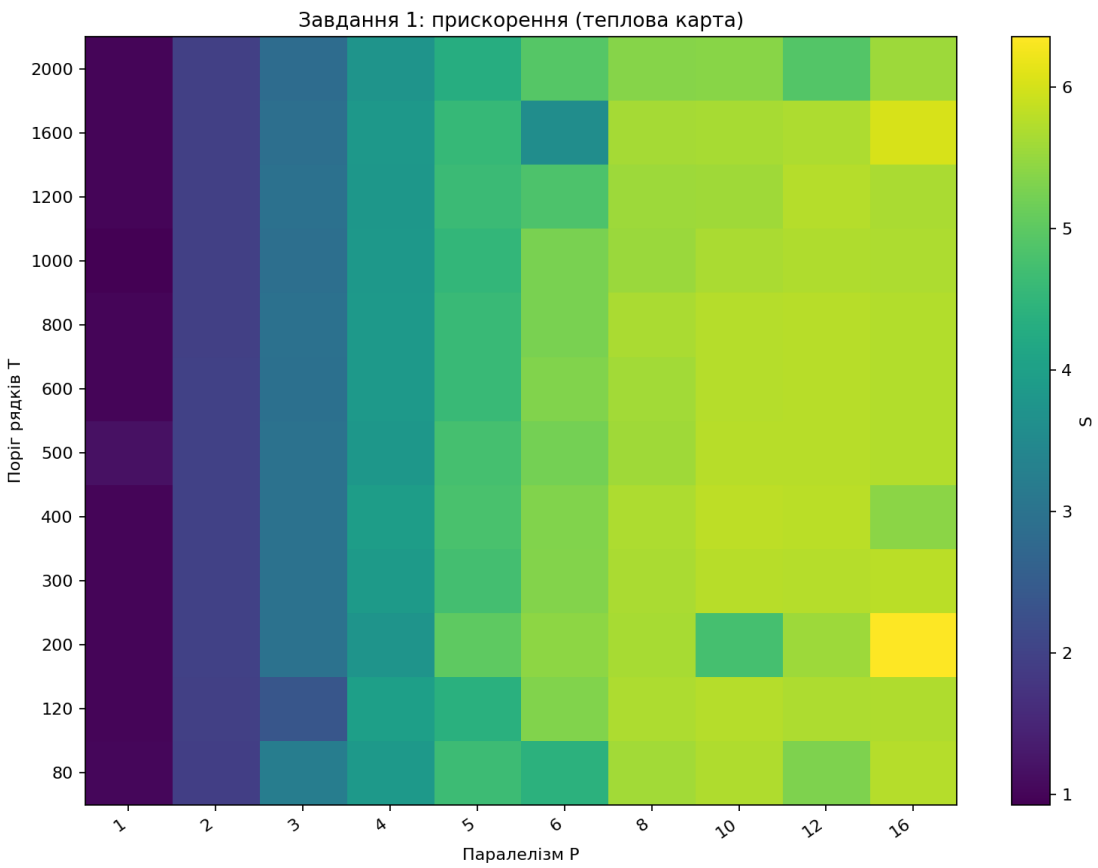
За гістограмою видно внесок коротких, середніх і рідкісних довгих слів; середнє та СКВ узгоджуються з формою розподілу. З графіків зазвичай видно реальне прискорення, але не лінійне до числа потоків: є витрати на fork/join і нерівномірність навантаження. Ефективність часто нижча за 1; завищений паралелізм на обмеженому залізі це підсилює.




Завдання 1: ефективність vs паралелізм
Ефективність E = S / P
Паралелізм P
T=80
T=120
T=200
T=300
T=400
T=500
T=600
T=800
T=1000
T=1200
T=1600
T=2000


Завдання 1: прискорення vs поріг рядків
Прискорення S
Поріг рядків T (листові задачі)
P=1
P=2
P=3
P=4
P=5
P=6
P=8
P=10
P=12
P=16

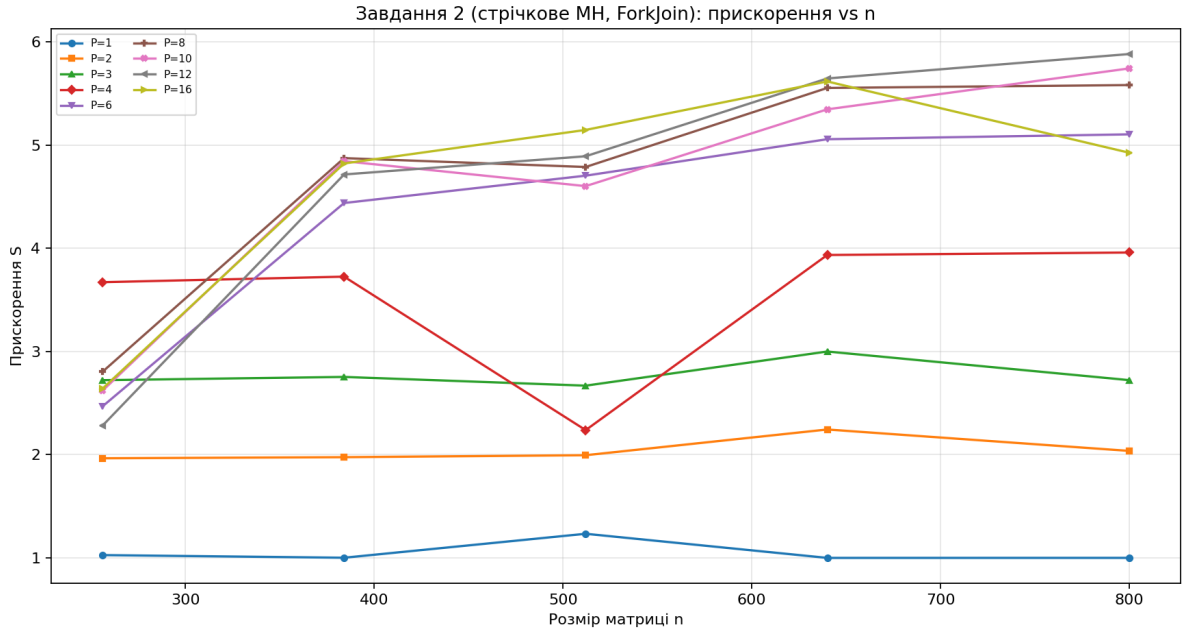

Завдання 1: час vs P (мінімальний поріг T = 80)
Середній час (мс)
Паралелізм P
Послідовно
ForkJoin

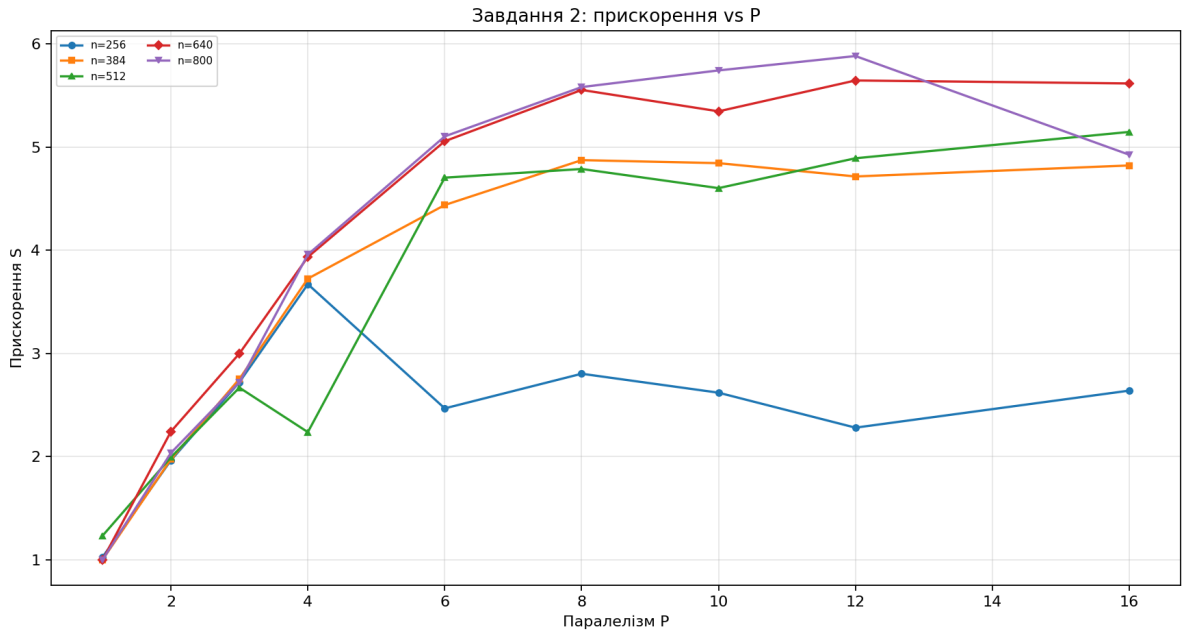

Завдання 1: прискорення (теплова карта)
Поріг рядків T
2000
1600
1200
1000
800
600
500
400
300
200
120
80
1
2
3
4
5
6
8
10
12
16
Паралелізм P
S
6
5
4
3
2
1

Завдання 2

Обрано стрічкове множення квадратних матриць C = A·B (той самий підхід, що й у практикумі з матриць): кожен рядок C будується окремо. Паралелізм fork/join зводиться до рекурсивного ділення діапазону рядків; у «листі» виконується звичайний обчислювальний фрагмент для свого блоку рядків. Послідовна версія з тією ж формулою дає еталон коректності й часу.

У експерименті змінюють розмір n і паралелізм P, час усереднюють після розігріву. Зазвичай прискорення краще на більших n, а зростання P дає виграш лише до межі, після якої домінують накладні витрати та властивості машини.





Завдання 3

Потрібні слова, що зустрічаються в кожному з набору текстових документів (перетин множин слів). Документи обробляються паралельно у сенсі fork/join: список файлів рекурсивно ділиться на дві частини, для кожної обчислюється перетин усередині піднабору, потім результати перетинаються між собою. У листі задачі читається один файл і будується множина його слів (за тим самим правилом токенізації, що й у текстових задачах практикуму). Послідовний перетин усіх множин слугує перевіркою коректності.

Завдання 4

Задано набір ключових слів з предметної області інформаційних технологій. Документ вважається релевантним, якщо в його тексті як окремі слова-токени зустрічається не менше ніж фіксована кількість різних ключів (правило узгоджується з реалізацією). Паралелізм fork/join зводиться до поділу списку файлів: у листі аналізується один файл, у внутрішніх вузлах об’єднуються списки шляхів, що пройшли фільтр. Послідовний прохід по тому ж каталогу дає той самий набір відповідей і перевіряє коректність.

Висновок

У ході практикуму опановано ForkJoinFramework як зручний спосіб виразити паралелізм задач із рекурсивним розбиттям та асоціативним зведенням результатів. На відміну від «ручного» керування потоками, модель fork/join дозволяє зосередитись на структурі підзадач і передати балансування навантаження робочому пулу, що особливо помітно на обчисленнях великої гранулярності (множення матриць) і на обробці багатьох текстових одиниць (рядки, файли).

Перше завдання показало, що паралельний збір статистик по тексту дає стійке прискорення, яке залежить від кількості потоків, розміру корпусу та вибору порогу «листя» дерева задач; ефективність зазвичай нижча за одиницю, що узгоджується з накладними витратами та обмеженнями реальної системи. Друге завдання підтвердило придатність fork/join для числового стрічкового множення матриць і продемонструвало зростання виграшу з розміром задачі при адекватній гранулярності блоку рядків.

Третє та четверте завдання ілюструють текстовий контекст: перетин множин слів і відбір документів за ключовими словами природно розпаралелюються по файлах (або по частинах списку файлів), а коректність забезпечується порівнянням із послідовним варіантом на тих самих даних. Це підкреслює важливість асоціативності операцій зведення (перетин множин, об’єднання списків збігів) для коректної роботи паралельної схеми.

Загалом практикум підсилює розуміння того, що паралелізм вигідний не «завжди»: він окупається, коли корисна робота достатньо велика порівняно з витратами на розбиття й синхронізацію, а структура даних дозволяє рівномірно розподілити навантаження. Отримані навички є базою для подальшого використання пулів потоків і fork/join у більших програмних системах.

Код програми

Посилання на GitHub репозиторій: https://github.com/sokolovgit/parallel-computing-technologies

```
=== lab4/common/Words.java ===
package lab4.common;

import java.io.IOException;
import java.io.UncheckedIOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.HashSet;
import java.util.Locale;
import java.util.Set;
import java.util.function.Consumer;
import java.util.function.IntConsumer;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/** Tokenization helpers shared across lab4 tasks. */
public final class Words {

    private static final Pattern TOKEN =
Pattern.compile("[\\p{L}\\p{M}\\p{N}]+");

    private Words() {}

    /** Invokes {@code onLength} for each word length in the line (Unicode
letters, marks, numbers). */
    public static void forEachWordLength(String line, IntConsumer
onLength) {
        if (line == null || line.isEmpty()) {
            return;
        }
        Matcher m = TOKEN.matcher(line);
        while (m.find()) {
            onLength.accept(m.group().length());
        }
    }

    /** Lowercase word tokens (same class as {@link #forEachWordLength}).
*/
    public static void forEachWord(String line, Consumer<String> onWord) {
        if (line == null || line.isEmpty()) {
            return;
        }
        Matcher m = TOKEN.matcher(line);
        while (m.find()) {
            onWord.accept(m.group().toLowerCase(Locale.ROOT));
        }
    }

    /** Усі різні слова з UTF-8 текстового файлу (нижній регістр). */
    public static Set<String> readWordSet(Path path) {
        try {
```

```
            HashSet<String> s = new HashSet<>();
            for (String line : Files.readAllLines(path,
StandardCharsets.UTF_8)) {
                forEachWord(line, s::add);
            }
            return s;
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }
}

=== lab4/task1/Task1Main.java ===
package lab4.task1;

import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.time.Instant;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.concurrent.ForkJoinPool;
import java.util.stream.Stream;
import lab4.common.Words;

/**
 * Завдання 1: статистика довжини слова (ForkJoin) + бенчмарк. Параметри
замірів зафіксовані в константах
 * для звіту; режим запуску – лише опційний аргумент.
 */
public final class Task1Main {

    /** Розігрів і кількість вимірів (як у методичці для лаб. 3). */
    private static final int WARMUP = 2;
    private static final int RUNS = 20;

    /** Скільки разів повторити кожен рядок малого корпусу (щоб замір був
«важким»). */
    private static final int REPEAT_SAMPLES = 8000;

    /**
     * Пороги рядків у листі ForkJoin (більше значень → густіша сітка на
графіках; час прогону зростає
     * пропорційно кількості комбінацій P×T).
     */
    private static final int[] SWEEP_THRESHOLD_SAMPLES = {
        40, 80, 120, 200, 300, 400, 600, 800, 1200, 1600
    };
    private static final int[] SWEEP_THRESHOLD_CORPUS = {
        80, 120, 200, 300, 400, 500, 600, 800, 1000, 1200, 1600, 2000
    };
```

```
    /** Паралелізм ForkJoinPool (є значення більші за кількість ядер – для
перевантаження у звіті). */
    private static final int[] SWEEP_PARALLELISM = {1, 2, 3, 4, 5, 6, 8,
10, 12, 16};

    private static final Path DIR_SAMPLES = Paths.get("data/samples");
    private static final Path DIR_CORPUS = Paths.get("data/corpus");

    private static final Path CSV_SAMPLES =
Paths.get("results/task1_benchmark.csv");
    private static final Path CSV_CORPUS =
Paths.get("results/task1_corpus_benchmark.csv");

    private static int loadedFileCount;

    public static void main(String[] args) throws Exception {
        Path base = Paths.get(System.getProperty("lab4.home",
".")).normalize();
        String mode = args.length == 0 ? "demo" : args[0];

        switch (mode) {
            case "demo" -> runDemo(base);
            case "bench" -> runBenchCsv(base, DIR_SAMPLES, REPEAT_SAMPLES,
SWEEP_THRESHOLD_SAMPLES, CSV_SAMPLES);
            case "bench-corpus" -> runBenchCsv(base, DIR_CORPUS, 1,
SWEEP_THRESHOLD_CORPUS, CSV_CORPUS);
            default -> {
                System.err.println("Usage: Task1Main [demo | bench |
bench-corpus]");
                System.err.println("  demo         – статистика + один
бенчмарк у консоль (" + DIR_SAMPLES + ")");
                System.err.println("  bench        – CSV " + CSV_SAMPLES +
" (repeat=" + REPEAT_SAMPLES + ")");
                System.err.println("  bench-corpus – CSV " + CSV_CORPUS +
" (repeat=1, " + DIR_CORPUS + ")");
                System.exit(1);
            }
        }
    }

    private static void runDemo(Path base) throws IOException {
        String[] raw = loadAllTxtUnder(base, DIR_SAMPLES);
        if (raw.length == 0) {
            System.err.println("No .txt under " +
base.resolve(DIR_SAMPLES));
            System.exit(1);
        }
        String[] lines = expandLines(raw, REPEAT_SAMPLES);
        int p = Runtime.getRuntime().availableProcessors();
        int threshold = 400;

        WordLengthStats fork = computeParallel(lines, p, threshold);
        WordLengthStats seq = computeSequential(lines);
        if (!statsRoughlyEqual(fork, seq)) {
            System.err.println("Warning: parallel vs sequential
mismatch.");
        }
        printStats("ForkJoin (correctness)", fork);
```

```
        System.out.println();
        System.out.println("--- Benchmark (P=" + p + ", threshold=" +
threshold + ", runs=" + RUNS + ") ---");
        BenchResult br = benchmark(lines, p, threshold);
        System.out.printf("Mean sequential: %.3f ms%n", br.tSeqMs);
        System.out.printf("Mean ForkJoin:   %.3f ms%n", br.tParMs);
        System.out.printf("Speedup S:       %.3f%n", br.speedup);
        System.out.printf("Efficiency E:    %.3f (P=%d)%n", br.efficiency,
p);
    }

    private static void runBenchCsv(Path base, Path dataDir, int repeat,
int[] thresholds, Path csvRel)
            throws IOException {
        String[] raw = loadAllTxtUnder(base, dataDir);
        if (raw.length == 0) {
            System.err.println("No .txt under " + base.resolve(dataDir));
            System.exit(1);
        }
        String[] lines = expandLines(raw, repeat);
        Path out = base.resolve(csvRel).normalize();
        Files.createDirectories(out.getParent());

        long wordCount = computeSequential(lines).wordCount();
        String jvm = Runtime.version().toString();
        String os = (System.getProperty("os.name", "") + " " +
System.getProperty("os.arch", "")).trim();
        String ts = Instant.now().toString();

        try (BufferedWriter w =
                Files.newBufferedWriter(
                        out, StandardCharsets.UTF_8,
StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING)) {
            w.write(

"task,parallelism,line_threshold,repeat_factor,num_files,num_lines_raw,num
_lines_bench,"
                            +
"word_count,t_seq_ms,t_par_ms,speedup,efficiency,runs,warmup,jvm,os,timest
amp");
            w.newLine();
            for (int t : thresholds) {
                for (int p : SWEEP_PARALLELISM) {
                    if (p < 1) {
                        continue;
                    }
                    BenchResult br = benchmark(lines, p, t);
                    w.write(
                            String.format(
                                    Locale.ROOT,

"task1,%d,%d,%d,%d,%d,%d,%d,%.6f,%.6f,%.6f,%.6f,%d,%d,%s,%s,%s",
                                    p,
                                    t,
                                    repeat,
                                    loadedFileCount,
                                    raw.length,
```

```
                                    lines.length,
                                    wordCount,
                                    br.tSeqMs,
                                    br.tParMs,
                                    br.speedup,
                                    br.efficiency,
                                    RUNS,
                                    WARMUP,
                                    csvEscape(jvm),
                                    csvEscape(os),
                                    csvEscape(ts)));
                    w.newLine();
                }
            }
        }
        System.out.println("Wrote " + out.toAbsolutePath());
    }

    private static String csvEscape(String s) {
        if (s.contains(",") || s.contains("\"") || s.contains("\n") ||
s.contains("\r")) {
            return "\"" + s.replace("\"", "\"\"") + "\"";
        }
        return s;
    }

    private record BenchResult(double tSeqMs, double tParMs, double
speedup, double efficiency) {}

    private static BenchResult benchmark(String[] benchLines, int
parallelism, int lineThreshold) {
        for (int w = 0; w < WARMUP; w++) {
            computeSequential(benchLines);
            computeParallel(benchLines, parallelism, lineThreshold);
        }
        long seqTotal = 0;
        for (int r = 0; r < RUNS; r++) {
            long t0 = System.nanoTime();
            computeSequential(benchLines);
            long t1 = System.nanoTime();
            seqTotal += t1 - t0;
        }
        long parTotal = 0;
        try (ForkJoinPool pool = new ForkJoinPool(parallelism)) {
            for (int r = 0; r < RUNS; r++) {
                long t0 = System.nanoTime();
                WordLengthStatsTask task =
                        new WordLengthStatsTask(benchLines, 0,
benchLines.length, lineThreshold);
                pool.invoke(task);
                long t1 = System.nanoTime();
                parTotal += t1 - t0;
            }
        }
        double tSeqNs = (double) seqTotal / RUNS;
        double tParNs = (double) parTotal / RUNS;
        double speedup = tParNs > 0 ? tSeqNs / tParNs : 0.0;
        double eff = parallelism > 0 ? speedup / parallelism : 0.0;
```

```
        return new BenchResult(tSeqNs / 1_000_000.0, tParNs / 1_000_000.0,
speedup, eff);
    }

    static WordLengthStats computeSequential(String[] lines) {
        WordLengthStats acc = new WordLengthStats();
        for (String line : lines) {
            Words.forEachWordLength(line, acc::addWord);
        }
        return acc;
    }

    static WordLengthStats computeParallel(String[] lines, int
parallelism, int lineThreshold) {
        try (ForkJoinPool pool = new ForkJoinPool(parallelism)) {
            WordLengthStatsTask task = new WordLengthStatsTask(lines, 0,
lines.length, lineThreshold);
            return pool.invoke(task);
        }
    }

    private static String[] expandLines(String[] base, int repeat) {
        int r = Math.max(1, repeat);
        if (base.length == 0 || r <= 1) {
            return base.clone();
        }
        String[] out = new String[base.length * r];
        for (int i = 0; i < out.length; i++) {
            out[i] = base[i % base.length];
        }
        return out;
    }

    private static boolean statsRoughlyEqual(WordLengthStats a,
WordLengthStats b) {
        if (a.wordCount() != b.wordCount()) {
            return false;
        }
        if (Double.compare(a.mean(), b.mean()) != 0) {
            return false;
        }
        return Math.abs(a.standardDeviation() - b.standardDeviation()) <
1e-9;
    }

    private static void printStats(String title, WordLengthStats s) {
        System.out.println(title);
        System.out.println("Word count: " + s.wordCount());
        System.out.printf("Mean length: %.6f%n", s.mean());
        System.out.printf("Std dev (population): %.6f%n",
s.standardDeviation());
        System.out.println("Histogram (length -> count); last bucket is
>=" + WordLengthStats.BUCKETS + ":");
        long[] h = s.histogramCopy();
        for (int len = 1; len < WordLengthStats.BUCKETS; len++) {
            long c = h[len - 1];
            if (c > 0) {
                System.out.println("  " + len + ": " + c);
```

```
            }
        }
        long overflow = h[WordLengthStats.BUCKETS - 1];
        if (overflow > 0) {
            System.out.println("  >=" + WordLengthStats.BUCKETS + ": " +
overflow);
        }
    }

    private static String[] loadAllTxtUnder(Path base, Path relativeDir)
throws IOException {
        List<Path> files = new ArrayList<>();
        Path root = base.resolve(relativeDir).normalize();
        if (Files.isDirectory(root)) {
            try (Stream<Path> walk = Files.walk(root)) {
                walk.filter(p -> Files.isRegularFile(p) &&
p.toString().toLowerCase(Locale.ROOT).endsWith(".txt"))
                        .forEach(files::add);
            }
        }
        Collections.sort(files);
        loadedFileCount = files.size();
        List<String> lines = new ArrayList<>();
        for (Path p : files) {
            lines.addAll(Files.readAllLines(p, StandardCharsets.UTF_8));
        }
        return lines.toArray(new String[0]);
    }
}

=== lab4/task1/WordLengthStats.java ===
package lab4.task1;

import java.util.Arrays;

/** Mergeable statistics for the random variable "word length
(characters)". */
public final class WordLengthStats {

    /** Buckets for lengths 1..(BUCKETS-1); last bucket aggregates length
>= BUCKETS. */
    public static final int BUCKETS = 65;

    private final long[] histogram;
    private long wordCount;
    private long sumLengths;
    private long sumSqLengths;

    public WordLengthStats() {
        this.histogram = new long[BUCKETS];
    }

    public void addWord(int length) {
        if (length <= 0) {
            return;
        }
        int idx = length < BUCKETS ? length - 1 : BUCKETS - 1;
        histogram[idx]++;
```

```
        wordCount++;
        sumLengths += length;
        sumSqLengths += (long) length * length;
    }

    public void merge(WordLengthStats other) {
        for (int i = 0; i < BUCKETS; i++) {
            histogram[i] += other.histogram[i];
        }
        wordCount += other.wordCount;
        sumLengths += other.sumLengths;
        sumSqLengths += other.sumSqLengths;
    }

    public long wordCount() {
        return wordCount;
    }

    /** Population mean E[X]. */
    public double mean() {
        if (wordCount == 0) {
            return Double.NaN;
        }
        return (double) sumLengths / wordCount;
    }

    /** Population standard deviation sqrt(E[X^2] - E[X]^2). */
    public double standardDeviation() {
        if (wordCount == 0) {
            return Double.NaN;
        }
        double m = mean();
        double meanSq = (double) sumSqLengths / wordCount;
        double v = meanSq - m * m;
        return v > 0 ? Math.sqrt(v) : 0.0;
    }

    public long[] histogramCopy() {
        return Arrays.copyOf(histogram, BUCKETS);
    }

    public static WordLengthStats merge(WordLengthStats a, WordLengthStats
b) {
        WordLengthStats out = new WordLengthStats();
        out.merge(a);
        out.merge(b);
        return out;
    }
}

=== lab4/task1/WordLengthStatsTask.java ===
package lab4.task1;

import java.util.concurrent.RecursiveTask;
import lab4.common.Words;

/** ForkJoin: split line range; merge {@link WordLengthStats} from
children. */
```

```
public final class WordLengthStatsTask extends
RecursiveTask<WordLengthStats> {

    private final String[] lines;
    private final int from;
    private final int to;
    private final int lineThreshold;

    public WordLengthStatsTask(String[] lines, int from, int to, int
lineThreshold) {
        this.lines = lines;
        this.from = from;
        this.to = to;
        this.lineThreshold = lineThreshold;
    }

    @Override
    protected WordLengthStats compute() {
        int n = to - from;
        if (n <= lineThreshold) {
            WordLengthStats acc = new WordLengthStats();
            for (int i = from; i < to; i++) {
                Words.forEachWordLength(lines[i], acc::addWord);
            }
            return acc;
        }
        int mid = from + n / 2;
        WordLengthStatsTask left = new WordLengthStatsTask(lines, from,
mid, lineThreshold);
        WordLengthStatsTask right = new WordLengthStatsTask(lines, mid,
to, lineThreshold);
        left.fork();
        WordLengthStats r = right.compute();
        WordLengthStats l = left.join();
        l.merge(r);
        return l;
    }
}

=== lab4/task2/StripedMatMulTask.java ===
package lab4.task2;

import java.util.concurrent.RecursiveAction;

/**
 * Стрічкове множення C = A·B: кожен рядок C – скалярний добуток рядка A
на стовпці B.
 * Рекурсивно ділить діапазон рядків [rowStart, rowEnd) доки не стане не
більшим за leafRows.
 */
public final class StripedMatMulTask extends RecursiveAction {

    private final double[][] a;
    private final double[][] b;
    private final double[][] c;
    private final int rowStart;
    private final int rowEnd;
    private final int leafRows;
```

```
    public StripedMatMulTask(double[][] a, double[][] b, double[][] c, int
rowStart, int rowEnd, int leafRows) {
        this.a = a;
        this.b = b;
        this.c = c;
        this.rowStart = rowStart;
        this.rowEnd = rowEnd;
        this.leafRows = leafRows;
    }

    @Override
    protected void compute() {
        int n = rowEnd - rowStart;
        if (n <= leafRows) {
            multiplyRowRange(rowStart, rowEnd);
            return;
        }
        int mid = rowStart + n / 2;
        StripedMatMulTask left = new StripedMatMulTask(a, b, c, rowStart,
mid, leafRows);
        StripedMatMulTask right = new StripedMatMulTask(a, b, c, mid,
rowEnd, leafRows);
        left.fork();
        right.compute();
        left.join();
    }

    private void multiplyRowRange(int rs, int re) {
        int dim = a.length;
        for (int i = rs; i < re; i++) {
            for (int j = 0; j < dim; j++) {
                double sum = 0.0;
                for (int k = 0; k < dim; k++) {
                    sum += a[i][k] * b[k][j];
                }
                c[i][j] = sum;
            }
        }
    }
}

=== lab4/task2/Task2Main.java ===
package lab4.task2;

import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.time.Instant;
import java.util.Locale;
import java.util.concurrent.ForkJoinPool;

/**
```

```
 * Завдання 2: стрічкове множення квадратних матриць (як у лаб. 3) через
ForkJoin; порівняння з послідовним
 * варіантом і експорт CSV для прискорення.
 */
public final class Task2Main {

    private static final int WARMUP = 2;
    private static final int RUNS = 20;

    /** Мінімальна кількість рядків у листі дерева ForkJoin. */
    private static final int LEAF_ROWS = 32;

    private static final int[] SWEEP_N = {256, 384, 512, 640, 800};
    private static final int[] SWEEP_P = {1, 2, 3, 4, 6, 8, 10, 12, 16};

    private static final int DEMO_N = 512;
    private static final Path CSV_OUT =
Paths.get("results/task2_benchmark.csv");

    public static void main(String[] args) throws Exception {
        Path base = Paths.get(System.getProperty("lab4.home",
".")).normalize();
        String mode = args.length == 0 ? "demo" : args[0];

        switch (mode) {
            case "demo" -> runDemo();
            case "bench" -> runBenchCsv(base);
            default -> {
                System.err.println("Usage: Task2Main [demo | bench]");
                System.exit(1);
            }
        }
    }

    private static void runDemo() {
        int n = DEMO_N;
        double[][] a = new double[n][n];
        double[][] b = new double[n][n];
        double[][] cSeq = new double[n][n];
        double[][] cPar = new double[n][n];
        fillDeterministic(a, 1L);
        fillDeterministic(b, 2L);

        sequentialMultiply(a, b, cSeq);
        int p = Runtime.getRuntime().availableProcessors();
        parallelMultiply(a, b, cPar, p, LEAF_ROWS);

        double err = maxAbsDiff(cSeq, cPar);
        System.out.println("Task2 demo: n=" + n + ", max |seq - fj| = " +
err);
        if (err > 1e-6) {
            System.err.println("Warning: possible numeric mismatch.");
        }

        System.out.println();
        System.out.println("--- Benchmark (n=" + n + ", P=" + p + ",
leafRows=" + LEAF_ROWS + ", runs=" + RUNS + ") ---");
        BenchResult br = benchmarkOnce(n, p, LEAF_ROWS);
```

```
        System.out.printf("Mean sequential: %.3f ms%n", br.tSeqMs);
        System.out.printf("Mean ForkJoin:   %.3f ms%n", br.tParMs);
        System.out.printf("Speedup S:       %.3f%n", br.speedup);
        System.out.printf("Efficiency E:    %.3f%n", br.efficiency);
    }

    private static void runBenchCsv(Path base) throws IOException {
        Path out = base.resolve(CSV_OUT).normalize();
        Files.createDirectories(out.getParent());

        String jvm = Runtime.version().toString();
        String os = (System.getProperty("os.name", "") + " " +
System.getProperty("os.arch", "")).trim();
        String ts = Instant.now().toString();

        try (BufferedWriter w =
                Files.newBufferedWriter(
                        out, StandardCharsets.UTF_8,
StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING)) {
            w.write(

"task,parallelism,leaf_rows,n,t_seq_ms,t_par_ms,speedup,efficiency,runs,wa
rmup,jvm,os,timestamp");
            w.newLine();

            for (int n : SWEEP_N) {
                int leaf = Math.min(LEAF_ROWS, Math.max(8, n / 16));
                for (int p : SWEEP_P) {
                    if (p < 1) {
                        continue;
                    }
                    BenchResult br = benchmarkOnce(n, p, leaf);
                    w.write(
                            String.format(
                                    Locale.ROOT,

"task2,%d,%d,%d,%.6f,%.6f,%.6f,%.6f,%d,%d,%s,%s,%s",
                                    p,
                                    leaf,
                                    n,
                                    br.tSeqMs,
                                    br.tParMs,
                                    br.speedup,
                                    br.efficiency,
                                    RUNS,
                                    WARMUP,
                                    csvEscape(jvm),
                                    csvEscape(os),
                                    csvEscape(ts)));
                    w.newLine();
                }
            }
        }
        System.out.println("Wrote " + out.toAbsolutePath());
    }

    private static String csvEscape(String s) {
```

```
        if (s.contains(",") || s.contains("\"") || s.contains("\n") ||
s.contains("\r")) {
            return "\"" + s.replace("\"", "\"\"") + "\"";
        }
        return s;
    }

    private record BenchResult(double tSeqMs, double tParMs, double
speedup, double efficiency) {}

    private static BenchResult benchmarkOnce(int n, int parallelism, int
leafRows) {
        double[][] a = new double[n][n];
        double[][] b = new double[n][n];
        double[][] c = new double[n][n];
        fillDeterministic(a, 1L);
        fillDeterministic(b, 2L);

        for (int w = 0; w < WARMUP; w++) {
            sequentialMultiply(a, b, c);
            parallelMultiply(a, b, c, parallelism, leafRows);
        }

        long seqTotal = 0;
        for (int r = 0; r < RUNS; r++) {
            long t0 = System.nanoTime();
            sequentialMultiply(a, b, c);
            long t1 = System.nanoTime();
            seqTotal += t1 - t0;
        }

        long parTotal = 0;
        try (ForkJoinPool pool = new ForkJoinPool(parallelism)) {
            for (int r = 0; r < RUNS; r++) {
                long t0 = System.nanoTime();
                StripedMatMulTask task = new StripedMatMulTask(a, b, c, 0,
n, leafRows);
                pool.invoke(task);
                long t1 = System.nanoTime();
                parTotal += t1 - t0;
            }
        }

        double tSeqNs = (double) seqTotal / RUNS;
        double tParNs = (double) parTotal / RUNS;
        double speedup = tParNs > 0 ? tSeqNs / tParNs : 0.0;
        double eff = parallelism > 0 ? speedup / parallelism : 0.0;
        return new BenchResult(tSeqNs / 1_000_000.0, tParNs / 1_000_000.0,
speedup, eff);
    }

    static void sequentialMultiply(double[][] a, double[][] b, double[][]
c) {
        int n = a.length;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                double sum = 0.0;
                for (int k = 0; k < n; k++) {
```

```
                    sum += a[i][k] * b[k][j];
                }
                c[i][j] = sum;
            }
        }
    }

    static void parallelMultiply(double[][] a, double[][] b, double[][] c,
int parallelism, int leafRows) {
        try (ForkJoinPool pool = new ForkJoinPool(parallelism)) {
            StripedMatMulTask task = new StripedMatMulTask(a, b, c, 0,
a.length, leafRows);
            pool.invoke(task);
        }
    }

    private static void fillDeterministic(double[][] m, long seed) {
        java.util.Random rnd = new java.util.Random(seed);
        for (double[] row : m) {
            for (int j = 0; j < row.length; j++) {
                row[j] = rnd.nextDouble();
            }
        }
    }

    private static double maxAbsDiff(double[][] x, double[][] y) {
        double max = 0.0;
        for (int i = 0; i < x.length; i++) {
            for (int j = 0; j < x[i].length; j++) {
                max = Math.max(max, Math.abs(x[i][j] - y[i][j]));
            }
        }
        return max;
    }
}

=== lab4/task3/CommonWordsTask.java ===
package lab4.task3;

import java.nio.file.Path;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.concurrent.RecursiveTask;
import lab4.common.Words;

/**
 * Перетин множин слів по документах: рекурсивно ділить список файлів, у
листі читає один файл;
 * внутрішні вузли повертають перетин множин зліва й справа.
 */
public final class CommonWordsTask extends RecursiveTask<Set<String>> {

    private final List<Path> files;
    private final int from;
    private final int to;

    public CommonWordsTask(List<Path> files, int from, int to) {
```

```
        this.files = files;
        this.from = from;
        this.to = to;
    }

    @Override
    protected Set<String> compute() {
        int span = to - from;
        if (span <= 0) {
            return Set.of();
        }
        if (span == 1) {
            return readWords(files.get(from));
        }
        int mid = from + span / 2;
        CommonWordsTask left = new CommonWordsTask(files, from, mid);
        CommonWordsTask right = new CommonWordsTask(files, mid, to);
        left.fork();
        Set<String> r = right.compute();
        Set<String> l = left.join();
        return intersect(l, r);
    }

    static Set<String> intersect(Set<String> a, Set<String> b) {
        if (a.isEmpty() || b.isEmpty()) {
            return Set.of();
        }
        if (a.size() <= b.size()) {
            HashSet<String> out = new HashSet<>(a);
            out.retainAll(b);
            return out;
        }
        HashSet<String> out = new HashSet<>(b);
        out.retainAll(a);
        return out;
    }

    static Set<String> readWords(Path p) {
        return Words.readWordSet(p);
    }
}

=== lab4/task3/Task3Main.java ===
package lab4.task3;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.Set;
import java.util.concurrent.ForkJoinPool;
import java.util.stream.Stream;

/**
```

```
 * Завдання 3: слова, спільні для всіх текстових документів у каталозі,
через ForkJoin (перетин множин).
 */
public final class Task3Main {

    private static final Path DATA_DIR = Paths.get("data/task3");

    public static void main(String[] args) {
        Path base = Paths.get(System.getProperty("lab4.home",
".")).normalize();
        String mode = args.length == 0 ? "demo" : args[0];
        if (!"demo".equals(mode)) {
            System.err.println("Usage: Task3Main [demo]");
            System.exit(1);
        }
        runDemo(base);
    }

    private static void runDemo(Path base) {
        List<Path> files = listTxtFiles(base.resolve(DATA_DIR));
        if (files.isEmpty()) {
            System.err.println("No .txt under " + base.resolve(DATA_DIR));
            System.exit(1);
        }

        Set<String> seq = sequentialCommon(files);
        int p = Runtime.getRuntime().availableProcessors();
        Set<String> par;
        try (ForkJoinPool pool = new ForkJoinPool(p)) {
            CommonWordsTask task = new CommonWordsTask(files, 0,
files.size());
            par = pool.invoke(task);
        }

        if (!seq.equals(par)) {
            System.err.println("Mismatch: sequential size " + seq.size() +
", parallel " + par.size());
            System.exit(1);
        }

        System.out.println("Files: " + files.size() + ", common distinct
words: " + par.size());
        System.out.println("ForkJoinPool parallelism: " + p);
        List<String> sample = new ArrayList<>(par);
        Collections.sort(sample);
        int show = Math.min(40, sample.size());
        System.out.println("Sample (" + show + " / " + sample.size() +
"):");
        for (int i = 0; i < show; i++) {
            System.out.println("  " + sample.get(i));
        }
    }

    private static List<Path> listTxtFiles(Path root) {
        List<Path> out = new ArrayList<>();
        if (!Files.isDirectory(root)) {
            return out;
        }
```

```
        try (Stream<Path> walk = Files.walk(root)) {
            walk.filter(p -> Files.isRegularFile(p) &&
p.toString().toLowerCase(Locale.ROOT).endsWith(".txt"))
                    .forEach(out::add);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        Collections.sort(out);
        return out;
    }

    static Set<String> sequentialCommon(List<Path> files) {
        if (files.isEmpty()) {
            return Set.of();
        }
        Set<String> acc = new
java.util.HashSet<>(CommonWordsTask.readWords(files.get(0)));
        for (int i = 1; i < files.size(); i++) {
            acc.retainAll(CommonWordsTask.readWords(files.get(i)));
            if (acc.isEmpty()) {
                break;
            }
        }
        return acc;
    }
}

=== lab4/task4/KeywordSearchTask.java ===
package lab4.task4;

import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Set;
import java.util.concurrent.RecursiveTask;
import lab4.common.Words;

/**
 * Паралельний відбір файлів, у тексті яких зустрічається достатньо
ключових слів з предметної області IT.
 */
public final class KeywordSearchTask extends RecursiveTask<List<Path>> {

    /** Ключові слова (нижній регістр), область «інформаційні технології».
*/
    static final Set<String> IT_KEYWORDS =
            Set.of(
                    "java",
                    "python",
                    "algorithm",
                    "database",
                    "network",
                    "thread",
                    "server",
                    "client",
                    "cloud",
                    "api",
```

```
                    "software",
                    "hardware",
                    "cpu",
                    "memory",
                    "cache",
                    "compiler",
                    "debug",
                    "repository",
                    "kubernetes",
                    "docker",
                    "linux",
                    "encryption",
                    "protocol");

    /** Мінімальна кількість різних ключових слів як цілих токенів у
документі. */
    static final int MIN_KEYWORD_HITS = 2;

    private final List<Path> files;
    private final int from;
    private final int to;

    public KeywordSearchTask(List<Path> files, int from, int to) {
        this.files = files;
        this.from = from;
        this.to = to;
    }

    @Override
    protected List<Path> compute() {
        int span = to - from;
        if (span <= 0) {
            return List.of();
        }
        if (span == 1) {
            Path p = files.get(from);
            Set<String> words = Words.readWordSet(p);
            if (countKeywordHits(words) >= MIN_KEYWORD_HITS) {
                return List.of(p);
            }
            return List.of();
        }
        int mid = from + span / 2;
        KeywordSearchTask left = new KeywordSearchTask(files, from, mid);
        KeywordSearchTask right = new KeywordSearchTask(files, mid, to);
        left.fork();
        List<Path> r = right.compute();
        List<Path> l = left.join();
        ArrayList<Path> out = new ArrayList<>(l.size() + r.size());
        out.addAll(l);
        out.addAll(r);
        Collections.sort(out);
        return out;
    }

    static int countKeywordHits(Set<String> docWords) {
        int n = 0;
        for (String kw : IT_KEYWORDS) {
```

```
            if (docWords.contains(kw)) {
                n++;
            }
        }
        return n;
    }

    static boolean matches(Set<String> docWords) {
        return countKeywordHits(docWords) >= MIN_KEYWORD_HITS;
    }
}

=== lab4/task4/Task4Main.java ===
package lab4.task4;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.concurrent.ForkJoinPool;
import java.util.stream.Stream;
import lab4.common.Words;

/**
 * Завдання 4: пошук .txt документів, що відповідають набору IT-ключових
слів (fork/join по списку файлів).
 */
public final class Task4Main {

    private static final Path DATA_DIR = Paths.get("data/task4");

    public static void main(String[] args) {
        Path base = Paths.get(System.getProperty("lab4.home",
".")).normalize();
        String mode = args.length == 0 ? "demo" : args[0];
        if (!"demo".equals(mode)) {
            System.err.println("Usage: Task4Main [demo]");
            System.exit(1);
        }
        runDemo(base);
    }

    private static void runDemo(Path base) {
        List<Path> files = listTxtFiles(base.resolve(DATA_DIR));
        if (files.isEmpty()) {
            System.err.println("No .txt under " + base.resolve(DATA_DIR));
            System.exit(1);
        }

        List<Path> seq = sequentialMatches(files);
        int p = Runtime.getRuntime().availableProcessors();
        List<Path> par;
        try (ForkJoinPool pool = new ForkJoinPool(p)) {
```

```
            KeywordSearchTask task = new KeywordSearchTask(files, 0,
files.size());
            par = pool.invoke(task);
        }

        Collections.sort(seq);
        if (!seq.equals(par)) {
            System.err.println("Mismatch sequential vs parallel.");
            System.err.println("seq: " + seq);
            System.err.println("par: " + par);
            System.exit(1);
        }

        List<String> kwPreview = new
ArrayList<>(KeywordSearchTask.IT_KEYWORDS);
        Collections.sort(kwPreview);
        int showKw = Math.min(10, kwPreview.size());
        System.out.println(
                "IT keyword set (total " + kwPreview.size() + ", show " +
showKw + "): "
                        + kwPreview.subList(0, showKw));
        System.out.println(
                "Rule: at least " + KeywordSearchTask.MIN_KEYWORD_HITS + "
distinct keywords as word tokens.");
        System.out.println("Files scanned: " + files.size() + ", matches:
" + par.size() + ", pool P=" + p);
        for (Path path : par) {
            int hits =
KeywordSearchTask.countKeywordHits(Words.readWordSet(path));
            System.out.println("  " + path.getFileName() + " (" + hits + "
keyword hits)");
        }
    }

    private static List<Path> listTxtFiles(Path root) {
        List<Path> out = new ArrayList<>();
        if (!Files.isDirectory(root)) {
            return out;
        }
        try (Stream<Path> walk = Files.walk(root)) {
            walk.filter(x -> Files.isRegularFile(x) &&
x.toString().toLowerCase(Locale.ROOT).endsWith(".txt"))
                    .forEach(out::add);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        Collections.sort(out);
        return out;
    }

    static List<Path> sequentialMatches(List<Path> files) {
        List<Path> out = new ArrayList<>();
        for (Path path : files) {
            if (KeywordSearchTask.matches(Words.readWordSet(path))) {
                out.add(path);
            }
        }
        return out;
```

```
    }
}
```